

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

Máximo Quiroga

TRABAJO PRÁCTICO N.º 1

**Del dominio al esquema: modelo entidad-relación, modelo relacional,
normalización y DDL**

Materia	Base de Datos 2
Carrera	Tecnicatura Universitaria en Programación a Distancia (UTN)
Trabajo práctico	N.º 1- Del dominio al esquema: modelo ER, modelo relacional, normalización y DDL
Proyecto integrador	Food Store (sistema de gestión de pedidos de un negocio de comidas) - pieza correspondiente a la Semana 1
Modalidad	
Fecha de entrega	27/8

1. Presentación y objetivos

Este trabajo práctico inaugura el hilo conductor de la materia: el proyecto integrador Food Store, un sistema de gestión de pedidos para un negocio de comidas. A partir de un enunciado en lenguaje natural, van a recorrer el camino completo que hace una base de datos relacional bien diseñada: analizar el dominio, modelarlo con un diagrama entidad-relación, derivar el esquema relacional correspondiente, verificar (y justificar) que ese esquema está normalizado, y finalmente escribir el DDL que lo crea en PostgreSQL.

Al finalizar este trabajo van a haber producido `schema.sql`, la primera pieza del proyecto integrador, que se irá ampliando semana a semana con datos, consultas, vistas, funciones y transacciones.

Al completar este TP vas a ser capaz de:

- Construir un modelo entidad-relación a partir de un enunciado, identificando entidades, atributos, claves, cardinalidades y participación (total o parcial) de cada relación.
- Derivar el modelo relacional correspondiente a un modelo ER, resolviendo relaciones 1:N mediante clave foránea y relaciones N:M mediante una tabla intermedia (entidad asociativa).
- Aplicar el proceso de normalización hasta 3FN/BCNF sobre un conjunto de datos, identificando dependencias funcionales y justificando por qué cada forma normal se cumple o se viola.
- Escribir DDL correcto en PostgreSQL: tipos de datos apropiados, claves primarias y foráneas, restricciones declarativas (NOT NULL, UNIQUE, CHECK, DEFAULT) e índices justificados por el uso esperado de las tablas.

2. El caso: Food Store

Un negocio de comidas quiere reemplazar sus planillas de cálculo por una base de datos. Así describe su operación el propietario:

El negocio, en sus palabras: <<Vendemos productos que agrupamos en categorías, como pizzas o bebidas. Los clientes se registran con sus datos y hacen pedidos. Cada pedido puede incluir varios productos, y cada producto se vende en muchos pedidos distintos. Necesitamos saber cuánto se vendió de cada línea, aunque el precio de un producto cambie con el tiempo: lo que se facturó en marzo no puede cambiar porque en abril subimos el precio.>>

Además de esa descripción general, la cátedra fija las siguientes reglas de negocio (restricciones del dominio) que el modelo debe respetar:

- **R1.** Todo producto pertenece exactamente a una categoría; una categoría puede tener muchos productos, o ninguno todavía (por ejemplo, una categoría recién creada).
- **R2.** Todo pedido pertenece exactamente a un cliente registrado; un cliente puede no haber hecho ningún pedido aún, o haber hecho varios.
- **R3.** Un pedido puede incluir varios productos distintos, y un mismo producto puede aparecer en muchos pedidos distintos (relación de muchos a muchos entre pedido y producto).
- **R4.** De cada producto dentro de un pedido interesa registrar cuántas unidades se pidieron y a qué precio unitario, porque el precio de lista de un producto puede cambiar con el tiempo y no debe alterar pedidos ya facturados.
- **R5.** Un producto no puede tener stock ni precio negativos.
- **R6.** Cada cliente tiene un correo electrónico que lo identifica de forma única en el sistema.
- **R7.** El negocio no elimina físicamente productos ni categorías dados de baja: prefiere conservarlos marcados como inactivos, para no perder el historial de pedidos que los

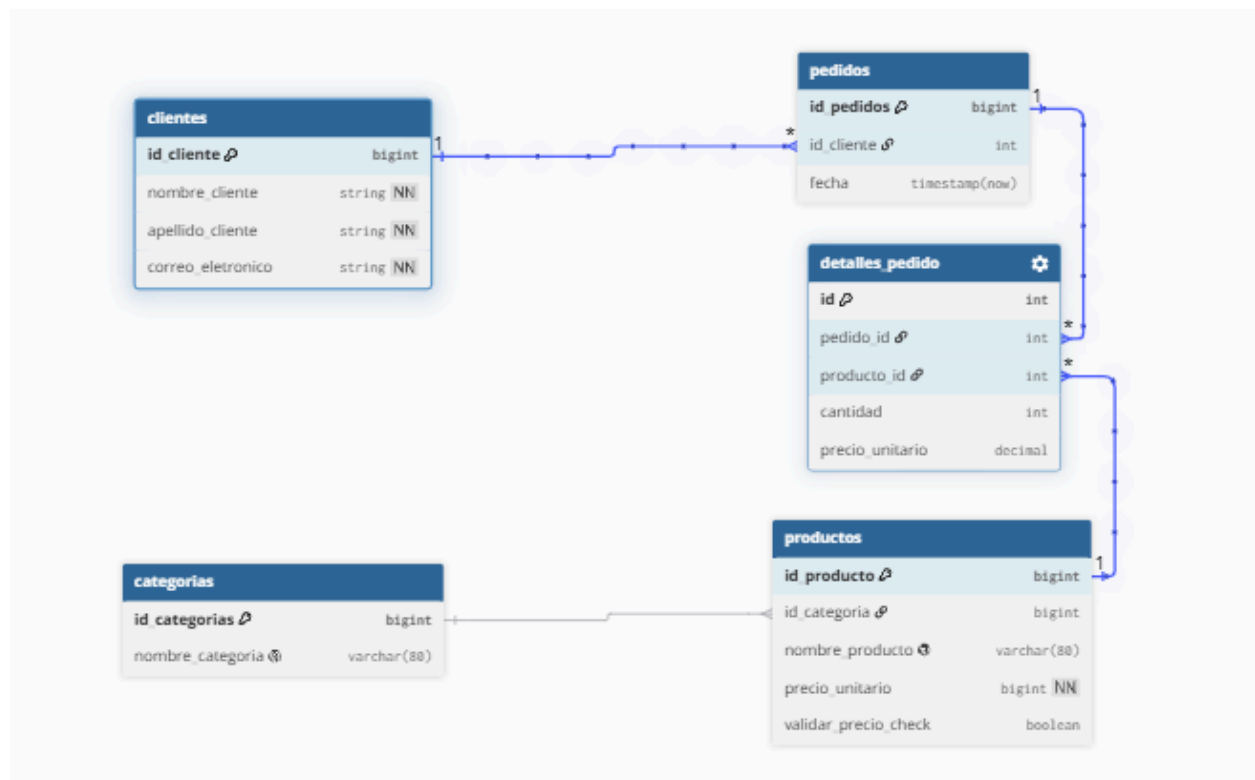
referencian.

Nota: estas reglas son la base para las cardinalidades, participaciones y restricciones que vas a modelar en cada parte. Se te pueden ocurrir atributos adicionales razonables (por ejemplo, teléfono del cliente, descripción del producto); si los agregas, déjalo aclarado en tu diccionario de datos.

3. Consigna

El trabajo se organiza en cuatro partes secuenciales: cada una toma como entrada el resultado de la anterior. Te recomendamos resolverlas en orden.

Parte 1 - Modelo entidad-relación (25%)

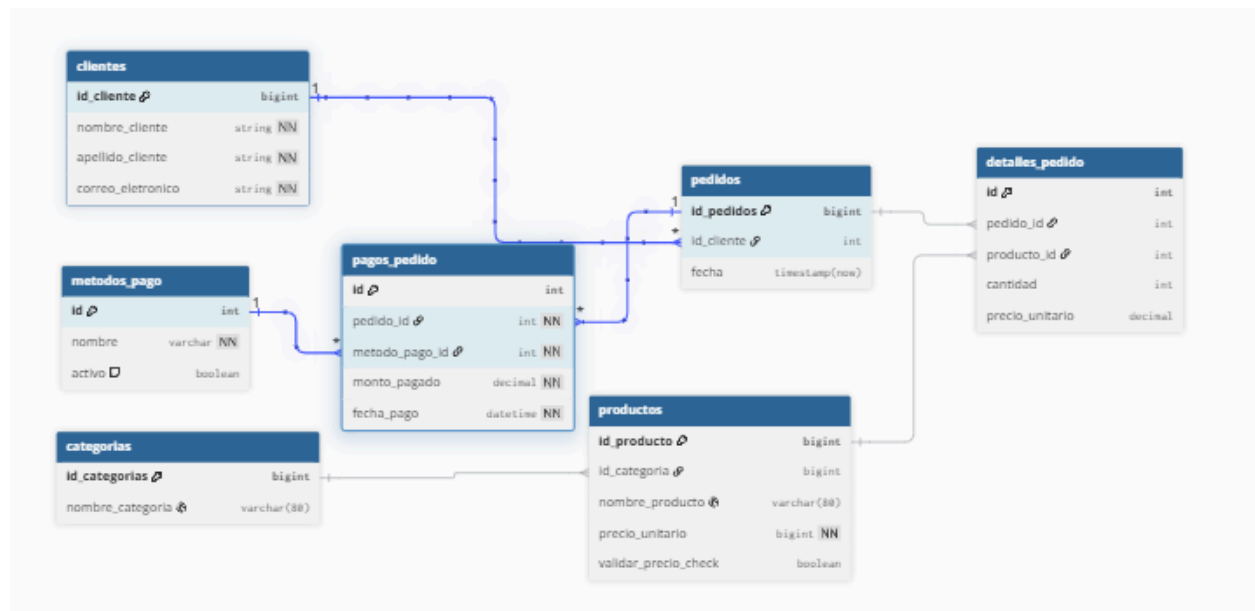


- Poner `producto_id` dentro de `pedidos`** : Esto significaría que un pedido **solo podría contener un único producto**. Si un cliente quiere comprar leche y pan, tendrías que registrar dos pedidos distintos, con fechas distintas y números de factura separados. Se perdería la noción de "carrito de compras" o "compra unificada".
- `productos` tiene participación TOTAL**: Todo producto que se ingresa a la tienda **debe** pertenecer obligatoriamente a una categoría para poder ser clasificado y facturado correctamente. No se permiten productos "huérfanos".

categorias tiene participación **PARCIAL**: Puedes crear una categoría nueva (por ejemplo, "Liquidación de Invierno" o "Tecnología") y tenerla vacía temporalmente sin que contenga ningún producto asociado todavía.

- c) En este modelo, lo ideal es **separarlo** en **nombre** y **apellido** por las siguientes razones:
 - **Búsquedas y filtros eficientes:** Si un administrador necesita buscar a todos los clientes de apellido "García", la base de datos resolverá la consulta muchísimo más rápido si busca en una columna dedicada exclusivamente a los apellidos.
 - **Personalización y Marketing:** Si envías correos automáticos de seguimiento de pedidos, es mucho más natural y profesional saludar con un *"Hola, Juan"* que con un *"Hola, Juan Carlos García"*.
 - **Ordenamiento:** Permite ordenar alfabéticamente a los clientes por su apellido de manera nativa en cualquier reporte.

Parte 2 - Derivación al modelo relacional (20%)



Parte 3 - Normalización hasta 3FN/BCNF (25%)

Food Store todavía registra sus ventas en una única planilla plana. Este es un extracto real de esa planilla (una fila por cada línea de producto vendido dentro de un pedido):

N.º pedido	Fecha	Cliente	Producto	Categoría	Precio unitario	Cant.	Subtotal	Forma de pago
------------	-------	---------	----------	-----------	-----------------	-------	----------	---------------

1	01/03/2026	Ana Gómez	Muzzarella	Pizzas	1000.00	2	2000.00	EFFECTIVO
1	01/03/2026	Ana Gómez	Coca 1.5L	Bebidas	800.00	1	800.00	EFFECTIVO
2	01/03/2026	Luis Paz	Napolitana	Pizzas	1500.00	1	1500.00	TARJETA
3	05/03/2026	Ana Gómez	Muzzarella	Pizzas	1050.00	3	3150.00	TRANSFERENCIA
4	06/03/2026	Marta Ruiz	Coca 1.5L	Bebidas	800.00	4	3200.00	EFFECTIVO
4	06/03/2026	Marta Ruiz	Napolitana	Pizzas	1500.00	2	3000.00	EFFECTIVO
5	07/03/2026	Luis Paz	Muzzarella	Pizzas	1050.00	1	1050.00	TARJETA

Las reglas de negocio proporcionadas ratifican formalmente el diseño relacional.

Teniendo en cuenta las reglas de la parte 1 y solo la planilla, esta fue mi normalizacion tratando de no dejarme llevar por completo por el ER, para que se pueda notar una verdadera diferencia

1. Clave candidata

La regla “*dentro de un mismo pedido, un producto no se repite en más de una línea*” garantiza unívocamente que la clave primaria de la tabla general es {N.º pedido, Producto}.

2. Dependencias Funcionales (DF)

DF1: N.º pedido -> Fecha, Cliente, Forma de pago (Sustentado por: “Cada pedido tiene una única fecha, un único cliente y una única forma de pago”).

DF2: Producto -> Categoría (Sustentado por: “Cada producto pertenece a una única categoría”).

DF3: {N.º pedido, Producto} -> Cant., Precio unitario (Sustentado por: “El precio de un producto puede variar en el tiempo”. El precio no depende estáticamente del producto, sino del producto en el contexto de un pedido específico).

DF4: {Precio unitario, Cant.} -> Subtotal (Dependencia matemática transitiva).

3. Primera Forma Normal (1FN)

La relación cumple con 1FN dado que todos los atributos contienen valores atómicos y no existen grupos repetidos dentro de las celdas.

4. Segunda Forma Normal (2FN)

Para cumplir 2FN, se eliminan las dependencias parciales (DF1 y DF2) donde atributos dependen solo de un fragmento de la clave compuesta.

Pedido: N.º pedido (PK), Fecha, Cliente, Forma de pago

Producto: Producto (PK), Categoría

Detalle_Pedido: N.º pedido (PK), Producto (PK), Precio unitario, Cant., Subtotal

5. Tercera Forma Normal (3FN)

Para cumplir 3FN, se elimina el atributo Subtotal de Detalle_Pedido porque representa una dependencia transitiva (DF4) y es un campo calculado a partir del precio y la cantidad.

Detalle_Pedido: N.º pedido (PK), Producto (PK), Precio unitario, Cant.

6. Forma Normal de Boyce-Codd (BCNF)

- El modelo final cumple con BCNF ya que en las tablas resultantes todo atributo que actúa como determinante es, de hecho, una clave candidata (N.º pedido en Pedido, Producto en Producto, y {N.º pedido, Producto} en Detalle_Pedido). No existen anomalías de actualización.

El modelo ER contiene múltiples atributos que no existían en la tabla de datos históricos: correo_eletronico y apellido_cliente (en clientes); monto_pagado y fecha_pago (en pagos_pedido); el estado activo (en metodos_pago) y validar_precio_check (en productos).

El diagrama ER implementa claves primarias subrogadas (IDs autoincrementales como id_cliente, id_categorias), mientras que la normalización de la planilla dependía de los nombres descriptivos como identificadores.

Parte 4 - DDL en PostgreSQL (25%)

Tablas						
>	categorias	8K				
>	clientes	16K				
>	detalles_pedido	8K				
>	pedidos	16K				
>	productos	16K				
>	Foreign Tables					

column Name	#	Tipo de datos	Identidad	Collation	No Nulo	Por defecto
id_categoria	1	int8	Always		[v]	
nombre	2	varchar(100)		default	[v]	
activo	3	bool			[v]	true
fecha_creacion	4	timestampz			[v]	now()

Script mejorado por ia, adjunto en la entrega del campus.

4. Formato y modalidad de entrega

- Modalidad: [a definir por la cátedra: individual o en grupos de hasta N integrantes].
- Entregables en un único archivo comprimido (.zip): diagrama ER (imagen o PDF), documento con el desarrollo de las Partes 2 y 3 (PDF o Word), y el archivo schema.sql de la Parte 4.
- Nombrá el archivo comprimido como Apellido_Nombre_TP1.zip (o Grupo_N_TP1.zip si es en grupo).
- Vía de entrega y fecha límite: [completar según el aula virtual de la cátedra].
- Se espera trabajo original. Se puede discutir el enfoque con compañeros, pero cada entrega (o grupo) debe redactar y resolver su propia versión.

5. Criterios de evaluación

La nota final del TP se compone según la siguiente rúbrica:

Criterio	Qué se evalúa	%
1. Modelo ER	Entidades y atributos correctos; claves primarias y candidatas bien identificadas; cardinalidades y participación (total/parcial) correctamente representadas y justificadas.	25

2. Modelo relacional	Cada entidad y relación del ER se traduce correctamente; las relaciones 1:N usan clave foránea en el lado N; la relación N:M se resuelve con tabla intermedia con su propia clave; tipos y nulabilidad coherentes con el ER.	20
3. Normalización	Dependencias funcionales identificadas correctamente; pasos 1FN -> 2FN -> 3FN/BCNF desarrollados y justificados (no solo enunciados); verificación explícita de BCNF.	25
4. DDL	Tipos de datos adecuados; PK y FK correctas; restricciones declarativas (NOT NULL, UNIQUE, CHECK, DEFAULT); al menos dos índices justificados; script ejecutable sin errores de sintaxis.	25
5. Integración y reflexión	Claridad y prolijidad de la presentación; respuestas fundamentadas a las preguntas de integración planteadas en la consigna.	5
Total		100

6. Recursos recomendados

- Herramientas de diagramación ER: dbdiagram.io, draw.io/ diagrams.net, Lucidchart, o papel y lápiz escaneado cualquiera es válida mientras el diagrama sea legible.
- Motor de base de datos: PostgreSQL (local, en un contenedor, o en un servicio como db-fiddle.com) con un cliente como pgAdmin, DBeaver o la terminal psql, para probar que

tu schema.sql se ejecuta sin errores.

- Material teórico visto en clase sobre modelo entidad-relación, reglas de derivación al modelo relacional, formas normales y sintaxis DDL de PostgreSQL.

7. La IA como motor primario en este TP

Como en todas las cátedras de la carrera, en este TP la inteligencia artificial no es una herramienta opcional de apoyo: es el motor primario con el que se produce el trabajo, con OpenCode y Kiro como herramientas transversales del segundo año. Esto no reemplaza tu criterio como responsable de cada script y cada decisión de diseño: lo pone a prueba en cada paso.

La IA como motor primario

- Puesta a punto del flujo del segundo año sobre datos: OpenCode y Kiro sobre un repositorio con los scripts de la base, y el protocolo de seguridad de la cátedra: copia, transacción, respaldo.
- Escenarios de concurrencia reconstruidos con IA: dado un conflicto entre transacciones, explicar qué pasó y qué nivel de aislamiento lo evita, verificando en el motor.
- El riesgo fundacional: por qué un script generado jamás se ejecuta sin leerse; casos reales de destrucción de datos por automatización acrítica.

Actividades prácticas con IA

- Laboratorio: reproducir anomalías de aislamiento con dos sesiones concurrentes sobre la base del proyecto; informe explicado con apoyo de IA y verificado.
- Primer ejercicio con OpenCode: scripts de restricciones e integridad versionados en Git, con revisión de diff y defensa.

Nota: los dos primeros puntos de esta sección (concurrencia y transacciones) anticipan contenidos que se profundizan en la Semana 2 de la cátedra; en este TP alcanza con dejar registrada la puesta a punto del flujo de IA y el primer ejercicio de integridad versionado en Git. Documentá todo en tu Declaración de Uso de IA (DUIA): herramienta usada, para qué, y con qué prompts o specs.

Anexo I - Glosario rápido

- **Entidad:** objeto o concepto del dominio sobre el que se necesita almacenar información (por ejemplo, producto).
- **Atributo:** propiedad o característica de una entidad (por ejemplo, precio).
- **Clave candidata:** conjunto mínimo de atributos que identifica de forma única a cada instancia de una entidad; puede haber más de una.
- **Clave primaria:** la clave candidata elegida para identificar cada fila de una tabla.
- **Clave foránea:** atributo (o conjunto de atributos) de una tabla que referencia la clave primaria de otra, materializando una relación.
- **Cardinalidad:** cuántas instancias de una entidad pueden asociarse con cuántas

instancias de otra en una relación (1:1, 1:N, N:M).

- **Participación total / parcial:** indica si toda instancia de una entidad debe obligatoriamente participar en una relación (total) o si puede existir sin participar (parcial).
- **Dependencia funcional (FD):** notación $A \rightarrow B$ que expresa que el valor de A determina de forma única el valor de B.
- **1FN:** todos los atributos tienen valores atómicos (sin listas ni grupos repetidos dentro de una misma celda).
- **2FN:** está en 1FN y, además, ningún atributo no clave depende de solo una parte de una clave compuesta.
- **3FN:** está en 2FN y, además, ningún atributo no clave depende transitivamente de la clave (a través de otro atributo no clave).
- **BCNF:** para toda dependencia funcional no trivial $X \rightarrow Y$, X es una clave candidata (una condición más estricta que 3FN).